

Explainable ML Tutorial

Jane Doe

2026-07-10

Table of contents

Preface	6
Notation	7
Numbers and Arrays	7
Sets and Graphs	7
Indexing	8
Linear Algebra Operations	8
Calculus	8
Probability and Information Theory	9
Functions	9
Datasets and Distributions	10
Reinforcement Learning	11
Multi-Agent Systems	11
Introduction	12
I Basics of Explainable Machine Learning	13
1 Introduction	14
1.1 Intrinsic vs. Post-Hoc Methods	14
1.1.1 Linear Regression	15
1.1.2 Logistic Regression	18
1.1.3 Decision Trees	18
1.1.4 K-Nearest Neighbors	18
1.2 Global vs. Local Explainability	18
1.3 Model-Agnostic vs. Model-Specific	18
1.4 Practical Considerations	18
1.4.1 Selecting an Explainability Method	18
1.4.2 Selecting an Explainability Method	18
1.4.3 Evaluating Explanations	18
1.4.4 Common Pitfalls	18
1.4.5 Best Practices	18
1.4.6 Summary	18

2	Taxonomy	19
2.1	Feature Attribution Methods	20
2.1.1	Introduction	20
2.1.2	Gradient-Based Attribution	20
2.1.3	Permutation Feature Importance	20
2.1.4	LIME	20
2.1.5	SHAP	20
2.1.6	Integrated Gradients	20
2.1.7	Grad-CAM	20
2.1.8	Comparison of Attribution Methods	20
2.2	Concept Bottleneck Methods	20
2.2.1	Introduction	20
2.2.2	Motivation	20
2.2.3	Model Architecture	20
2.2.4	Training Procedure	20
2.2.5	Applications	20
2.2.6	Advantages	20
2.2.7	Limitations	20
3	Method(s) Page 1	21
4	Method(s) Page 2	22
5	...	23
6	Applications	24
7	Conclusion	25
II	Explainability for Language Models	26
8	Introduction	27
8.1	Activation-based explanations	27
8.2	Concept-based explainability	27
8.2.1	Concepts and concept space	28
8.2.2	Notation	28
8.3	Concept discovery	29
9	Taxonomy	30
9.1	Activation-Based Explanations	30
9.2	Concept-Based Explanations	30
9.2.1	Preliminaries and Notation	30
9.2.2	Human-Defined Concepts (Supervised)	30

9.3	Concept Discovery	31
9.4	Mechanistic Interpretability	31
9.5	Attention-Visualization Methods	31
10	Method A: Concept Bottleneck Models	32
10.1	Architecture and inference	32
10.2	What does a CBM explain?	33
10.3	Training regimes	34
10.4	Concept interventions	35
10.5	Evaluating a CBM	35
10.6	Limitations and appropriate use cases	36
10.7	Reducing the need for concept annotations	37
11	Method(s) Page 2	38
12	...	39
13	Applications	40
14	Conclusion	41
III	Explainable Reinforcement Learning	42
15	Introduction to Explainable RL	43
15.1	Foundations	43
15.2	Challenges in XRL	44
16	Taxonomy	45
16.1	Feature importance (or: action level)	45
16.2	Learning process and MDP level (or: sequency level)	46
16.3	Policy level	46
16.4	Alternative taxonomy structure:	47
16.4.1	Visualization-Based Approaches	47
16.4.2	More stuff to sort in	47
16.5	Representation space analysis	47
16.6	Explaining decisions through critical states	47
16.7	Trajectory-based methods	47
16.8	Explainability via counterfactual reasoning	48
16.9	Evaluation of XRL methods	48
16.10	Applications	48
17	Perturbation-Based Saliency Maps for RL	49
	Introduction	49

Method: Perturbation-based saliency maps	49
Tradeoffs	50
Related	50
18 Method(s) Page 2	51
19 ...	52
20 Applications	53
21 Conclusion	54
 IV Explainability in Multi-Agent Systems	 55
22 Introduction	56
23 Taxonomy	57
23.1 What are agents?	57
23.2 What are multi-agent systems?	58
23.3 Technical Approaches and Methodologies	58
24 Method(s) Page 1	59
25 Method(s) Page 2	60
26 ...	61
27 Applications	62
28 Conclusion	63
References	64

Preface

Acknowledgements etc.

Notation

Here, we introduce the notation used consistently throughout this book. We follow the [Deep Learning Book](#) wherever possible.

Numbers and Arrays

Symbol	Description
a	A scalar (integer or real)
$\mathbf{a}$	A vector
A	A matrix
$\mathbf{A}$	A tensor
I_n	Identity matrix with n rows and n columns
I	Identity matrix with dimensionality implied by context
$e^{(i)}$	Standard basis vector $[0, \dots, 0, 1, 0, \dots, 0]$ with a 1 at position i
$\text{diag}(a)$	A square, diagonal matrix with diagonal entries given by a
a	A scalar random variable
$\mathbf{a}$	A vector-valued random variable
$\mathbf{A}$	A matrix-valued random variable

Sets and Graphs

Symbol	Description
$\mathbb{A}$	A set
$\mathbb{R}$	The set of real numbers
$\{0, 1\}$	The set containing 0 and 1
$\{0, 1, \dots, n\}$	The set of all integers between 0 and n
$[a, b]$	The real interval including a and b

Symbol	Description
$(a, b]$	The real interval excluding a but including b
$\mathcal{A} \setminus \mathcal{B}$	Set subtraction, i.e., the set containing the elements of $\mathcal{A}$ that are not in $\mathcal{B}$
$\mathcal{G}$	A graph
$Pa_{\mathcal{G}}(\mathbf{x}_i)$	The parents of $\mathbf{x}_i$ in $\mathcal{G}$

Indexing

Symbol	Description
a_i	Element i of vector a , with indexing starting at 1
a_{-i}	All elements of vector a except for element i
$A_{i,j}$	Element i, j of matrix A
$A_{i,:}$	Row i of matrix A
$A_{:,i}$	Column i of matrix A
$A_{i,j,k}$	Element (i, j, k) of a 3-D tensor $\mathbf{A}$
$\mathbf{A}_{:, :, i}$	2-D slice of a 3-D tensor
$\mathbf{a}_i$	Element i of the random vector $\mathbf{a}$

Linear Algebra Operations

Symbol	Description
$A^\top$	Transpose of matrix A
A^+	Moore-Penrose pseudoinverse of A
$A \odot B$	Element-wise (Hadamard) product of A and B
$\det(A)$	Determinant of A

Calculus

Symbol	Description
$\frac{dy}{dx}$	Derivative of y with respect to x

Symbol	Description
$\frac{\partial y}{\partial x}$	Partial derivative of y with respect to x
$\nabla_x y$	Gradient of y with respect to x
$\nabla_X y$	Matrix derivatives of y with respect to X
$\nabla_{\mathbf{x}} y$	Tensor containing derivatives of y with respect to $\mathbf{X}$
$\frac{\partial f}{\partial x}$	Jacobian matrix $J \in \mathbb{R}^{m \times n}$ of $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$
$\nabla_x^2 f(x)$ or $H(f)(x)$	The Hessian matrix of f at input point x
$\int f(x) dx$	Definite integral over the entire domain of x
$\int_S f(x) dx$	Definite integral with respect to x over the set S

Probability and Information Theory

Symbol	Description
$a \perp b$	The random variables a and b are independent
$a \perp b \mid c$	They are conditionally independent given c
$P(a)$	A probability distribution over a discrete variable
$p(a)$	A probability distribution over a continuous variable, or over a variable whose type has not been specified
$a \sim P$	Random variable a has distribution P
$\mathbb{E}_{x \sim P}[f(x)]$ or $\mathbb{E}f(x)$	Expectation of $f(x)$ with respect to $P(x)$
$\text{Var}(f(x))$	Variance of $f(x)$ under $P(x)$
$\text{Cov}(f(x), g(x))$	Covariance of $f(x)$ and $g(x)$ under $P(x)$
$H(x)$	Shannon entropy of the random variable x
$D_{\text{KL}}(P \parallel Q)$	Kullback-Leibler divergence of P and Q
$\mathcal{N}(x; \mu, \Sigma)$	Gaussian distribution over x with mean μ and covariance Σ

Functions

Symbol	Description
$f : \mathbb{A} \rightarrow \mathbb{B}$	The function f with domain $\mathbb{A}$ and range $\mathbb{B}$

Symbol	Description
$f \circ g$	Composition of the functions f and g
$f(x; \theta)$	A function of x parametrized by θ . (Sometimes we write $f(x)$ and omit the argument θ to lighten notation)
$\mathcal{L}(\theta)$	The loss function to be minimized
$\nabla_{\theta} \mathcal{L}$	The gradient of the loss with respect to parameters θ
$\log x$	Natural logarithm of x
$\sigma(x)$	Logistic sigmoid, $\frac{1}{1+\exp(-x)}$
$\zeta(x)$	Softplus, $\log(1 + \exp(x))$
$\ x\ _p$	L^p norm of x
$\ x\ _2$	L^2 norm of x
x^+	Positive part of x , i.e., $\max(0, x)$
$\mathbf{1}_{\text{condition}}$	is 1 if the condition is true, 0 otherwise

Sometimes we use a function f whose argument is a scalar but apply it to a vector, matrix, or tensor: $f(x)$, $f(X)$, or $f(\mathbf{X})$. This denotes the application of f to the array element-wise. For example, if $\mathbf{C} = \sigma(\mathbf{X})$, then $C_{i,j,k} = \sigma(X_{i,j,k})$ for all valid values of i , j and k .

Datasets and Distributions

Symbol	Description
$\mathcal{D}$	A dataset containing training or evaluation examples
$\mathbf{x}^{(i)}, y^{(i)}$	The i -th input feature vector and its corresponding target/label
p_{data}	The data generating distribution
$\hat{p}_{\text{data}}$	The empirical distribution defined by the training set
$\mathbb{X}$	A set of training examples
$x^{(i)}$	The i -th example (input) from a dataset
$y^{(i)}$ or $y^{(i)}$	The target associated with $x^{(i)}$ for supervised learning
X	The $m \times n$ matrix with input example $x^{(i)}$ in row $X_{i,:}$

Reinforcement Learning

Symbol	Description
$\mathcal{S}, \mathcal{A}$	The set of all possible states and actions
s_t, a_t, r_t	State, action, and reward at time step t
$P(s_{t+1} s_t, a_t)$	Transition dynamics: probability of reaching s_{t+1} given s_t and a_t
$\mathcal{R}(s_t, a_t)$	The reward function
γ	Discount factor, $\gamma \in [0, 1]$
$\pi(a s)$	A stochastic policy: probability of taking action a in state s
$V^\pi(s)$	The state-value function under policy π
$Q^\pi(s, a)$	The action-value (Q) function under policy π

Multi-Agent Systems

Symbol	Description
N	Total number of agents in the environment
i	Index of a specific agent, $i \in \{1, \dots, N\}$
$\mathbf{a} = \langle a_1, \dots, a_N \rangle$	Joint action taken by all agents
$\pi = \langle \pi_1, \dots, \pi_N \rangle$	Joint policy of all agents
$-i$	Subscript denoting all agents <i>except</i> agent i (e.g., $\mathbf{a}_{-i}$)

Introduction

Welcome to the tutorial!

Part I

Basics of Explainable Machine Learning

1 Introduction

(This section was previously called: “Categorization of XML Methods”)

There are many taxonomies in which interpretability methods can be categorized, they can be categorized according to the nature of the learning algorithm or the stage at which the model becomes interpretable into Intrinsic and Post-Hoc methods, They can also be categorized into Local and Global methods according to the scope of explanations.

1.1 Intrinsic vs. Post-Hoc Methods

Intrinsic methods indicate that the model itself is inherently interpretable by design the algorithm itself tends to be less complex and the factors affecting the decision making process are always disentangled (Molnar 2025). Linear Regression, Logistic Regression, Generalized linear models (GLMs) (Jingyuan Wang and Ji 2025) and Decision Trees are examples of models that are interpretable by design, they either show the importance of each feature, or come up with human readable decision rules (Miller 2019; Molnar 2025). While those models are extremely simple, efficient and interpretable they face many limitations when deployed to the real world, For example linear models can be interpreted by measuring the feature importance; by looking into the coefficient of each feature in the weight matrix (Molnar 2025). However, they are built on very strict assumptions and inductive biases such as linearity, independence, and absence of heteroscedasticity which are difficult to gurantee in the real world (Hastie, Tibshirani, and Friedman 2009), this limits the ability of such algorithms to capture true, non-linear patterns underlying the data or even model complex high dimensional data which limits the predictive performance of such models.

While linear models fail to model non-linear data, tree-based models come into play. They work by splitting the data multiple times according to discovered thresholds over the values of the features (Molnar 2025). Each learning algorithm uses a different criterion to grow the optimal tree. For example, ID3 measures the Information Gain criterion to find the best splits (Quinlan 1986), While CART optimizes for the Gini Impurity (Breiman et al. 1984). All tree based algorithms are capable of modeling non-linear data. Those models are Interpretable intrinsically; you can traverse the tree starting from the root to the leaf in order to find the decision in the form of sequential If-Else statements. However, due to the greedy nature of the learning algorithm it tends to increase the depth of the tree drastically leading to a trade-off between error minimization and being easily interpretable (Molnar 2025).

On the other hand, Post-Hoc methods work by reporting how an already trained model produces its predictions. For any given input it tries to provide understandable explanations for the learned decision boundaries by the more complex black-box models Molnar (2025). Post-Hoc methods can be categorized into two main categories: Model-Agnostic methods, and model-specific methods (Arrieta et al. 2019), Where the former can be plugged into any model to explain its decision space, e.g perturbation based approaches like LIME (Ribeiro, Singh, and Guestrin 2016) and SHAP (Lundberg and Lee 2017) that predict how the decision space changes when the input data point is perturbed, While the latter is specific to a model architecture like analyzing attention maps in transformer networks, or gradient-based methods that look into the gradient of the input with respect to the loss function to provide pixel-level explanations like Saliency Maps (Simonyan, Vedaldi, and Zisserman 2014), class activation maps (CAM) (Minh 2023) and it variants like GradCAMs (Selvaraju et al. 2019) in convolutional neural networks (CNNs) are also used to quantify the amount of input activation conditioned on the output class to provide a heat-map for the relevant regions for the predicted class.

1.1.1 Linear Regression

Linear regression represents the output variable y as a linear combination of the input features $\mathbf{X}$ weighted by the scaling coefficient matrix β . Mathematically:

$$\mathbf{y} = \mathbf{X}\beta + \varepsilon,$$

$$\mathbf{X} \in \mathbb{R}^{n \times p}, \quad \mathbf{y} \in \mathbb{R}^n, \quad \beta \in \mathbb{R}^p, \quad \varepsilon \in \mathbb{R}^n.$$

where ε is called the irreducible error term, it is typically considered a random variable with zero mean and constant variance. It captures the noise, and the variations in the data that the model failed to explain using the given features, n is the number of training instances, and p is the number of features.

We can run iterative optimization using standard gradient descent algorithm (Ruder 2017) to find the betas that minimize mean squared error (MSE):

$$J(\beta) = \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\beta\|_2^2.$$

$$\beta^{(t+1)} = \beta^{(t)} - \eta \frac{1}{n} \mathbf{X}^\top (\mathbf{X}\beta^{(t)} - \mathbf{y}),$$

$$\frac{\partial y}{\partial x_j} = \beta_j,$$

Linear Regression models are inherently interpretable through their coefficient vector β . Each coefficient β_j quantifies the effect of the corresponding feature represented by the j -th column of the feature matrix $\mathbf{X}$ as shown in Figure 1.1.

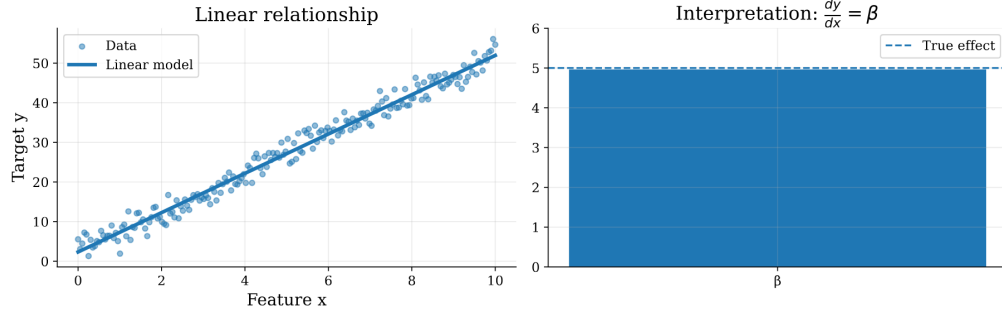


Figure 1.1: Linear regression correctly captures the relationship between the feature and the target. The coefficient β corresponds to the constant marginal effect $\frac{\partial y}{\partial x}$, making the model directly interpretable.

While Linear Regression is simple, and interpretable by design, many limitations rise from the implicit inductive biases assumed by this model (Jingyuan Wang and Ji 2025). First, Linear Regression models assumes a linear relationship between the input features and the target variable if the relation is not linear the learned coefficient fails to explain the effect of the feature as shown in Figure 1.2, more formally:

$$\frac{\partial y}{\partial x_j} \neq \beta_j,$$

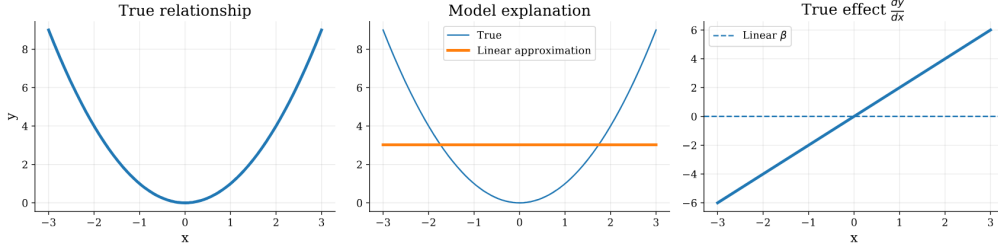


Figure 1.2: When the underlying data distribution is non-linear, linear regression coefficients fails to explain the relationship.

Also, Linear Regression models assumes independence between features. Feature correlation can be reflected in the Gram matrix $\mathbf{X}^\top \mathbf{X}$, where off-diagonal elements quantify the similarity between feature columns. Strong correlations between features introduce multicollinearity, making the estimated coefficients β unstable and reduces interpretability as shown in Figure 1.3.

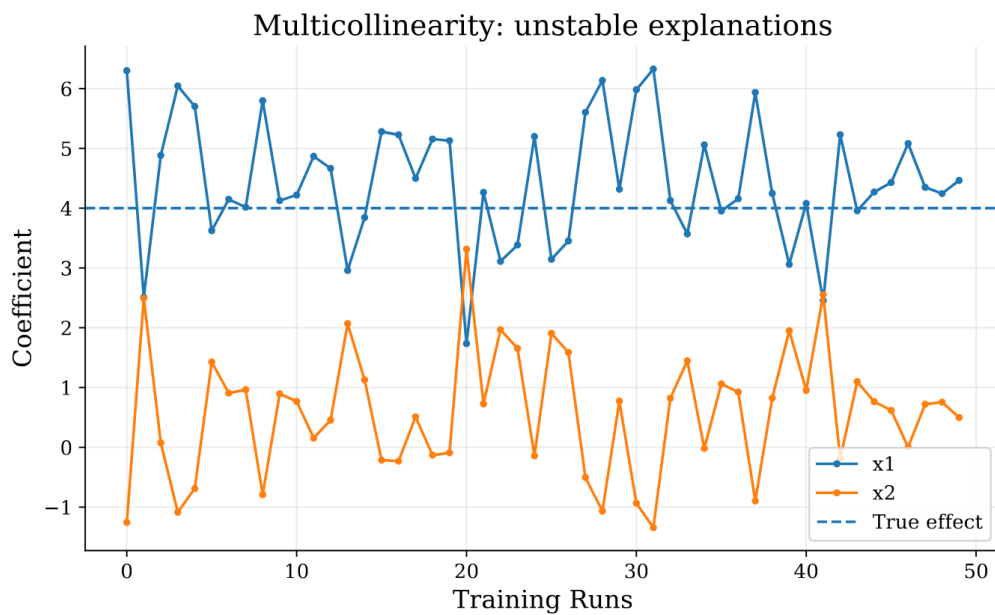


Figure 1.3: When features are mutually collinear the explanations provided by learned coefficients becomes unstable through multiple runs. In this example we generate a correlated feature using $x_2 = x_1 + \epsilon$, where $\epsilon_i \sim \mathcal{N}(0, 0.05^2)$.

1.1.2 Logistic Regression

1.1.3 Decision Trees

1.1.4 K-Nearest Neighbors

1.2 Global vs. Local Explainability

1.3 Model-Agnostic vs. Model-Specific

1.4 Practical Considerations

1.4.1 Selecting an Explainability Method

1.4.2 Selecting an Explainability Method

1.4.3 Evaluating Explanations

1.4.3.1 Fidelity

1.4.3.2 Robustness

1.4.3.3 Stability

1.4.3.4 Human Interpretability

1.4.4 Common Pitfalls

1.4.5 Best Practices

1.4.6 Summary

2 Taxonomy

2.1 Feature Attribution Methods

2.1.1 Introduction

2.1.2 Gradient-Based Attribution

2.1.3 Permutation Feature Importance

2.1.4 LIME

2.1.5 SHAP

2.1.5.1 Shapley Values

2.1.5.2 TreeSHAP

2.1.5.3 KernelSHAP

2.1.6 Integrated Gradients

2.1.7 Grad-CAM

2.1.8 Comparison of Attribution Methods

2.2 Concept Bottleneck Methods

2.2.1 Introduction

2.2.2 Motivation

2.2.3 Model Architecture

2.2.4 Training Procedure

2.2.5 Applications

2.2.6 Advantages

2.2.7 Limitations

3 Method(s) Page 1

4 Method(s) Page 2

5 ...

6 Applications

7 Conclusion

Part II

Explainability for Language Models

8 Introduction

Language models transform tokens through a sequence of learned representations before producing an output. Explainability methods study different parts of this process. Some inspect internal activations, some describe predictions using human-readable concepts, and others discover recurring patterns in the model’s representations.

This tutorial separates these approaches into three families. The boundary is useful for organizing the methods, although some techniques combine ideas from more than one family.

8.1 Activation-based explanations

Activation-based explanations examine the values produced inside a model as it processes an input. They may study individual units, compare activation patterns across examples, or identify directions in the representation space that are associated with a behavior. These methods work with the model’s internal representation and do not require the final explanation to use a predefined concept vocabulary.

8.2 Concept-based explainability

Concept-based explanations describe model behavior using properties that people can name and understand. For text, examples include *contains a negation*, *mentions poor service*, or *uses positive language*. A useful concept should have a clear meaning, be observable from the input, and be relevant to the task.

In supervised concept-based methods, the concept vocabulary is selected before training. Domain experts or dataset designers choose the concepts, and training examples are annotated with both task labels and concept values. This gives the explanation a fixed vocabulary, but its quality depends on whether that vocabulary is complete and consistently annotated.

8.2.1 Concepts and concept space

A concept is a property of an input. For this tutorial, we use $c_j^*(x)$ as an idealized value of concept j for input x :

$$c_j^* : \mathcal{X} \rightarrow \mathcal{C}_j, \quad (8.1)$$

where $\mathcal{X}$ is the input space and $\mathcal{C}_j$ is the set of possible values for concept j . A binary concept such as *contains a negation* has $\mathcal{C}_j = \{0, 1\}$. An ordinal concept such as sentiment strength could take values in $\{0, 1, 2, 3\}$. Concepts may also be categorical or continuous.

For k concepts, the concept space is

$$\mathcal{C} = \mathcal{C}_1 \times \cdots \times \mathcal{C}_k. \quad (8.2)$$

An input x is described by the concept vector

$$\mathbf{c}^*(x) = (c_1^*(x), \dots, c_k^*(x)) \in \mathcal{C}. \quad (8.3)$$

This is a modeling convention, not a claim that every concept has one objectively correct value. Real annotations are less certain. Annotators may disagree, a concept may be subjective, or the input may not contain enough information to determine its value. An observed annotation c_j should therefore be treated as a measurement of the intended concept, not necessarily as perfect ground truth. When uncertainty matters, a model can predict a distribution $p(c_j \mid x)$ instead of a single value.

8.2.2 Notation

Symbol	Meaning
$x \in \mathcal{X}$	Input example
$y \in \mathcal{Y}$	Task label
$c_j^*(x)$	True value of concept j
c_j	Annotated value of concept j
$\hat{c}_j$	Model-predicted value of concept j
$\mathbf{c}, \hat{\mathbf{c}}$	Annotated and predicted concept vectors
g_θ	Concept predictor
f_ϕ	Task predictor
$\hat{y}$	Model prediction

The distinction between c_j^* , c_j , and $\hat{c}_j$ matters. The true property may be uncertain, its annotation may be wrong, and the model may make another error when predicting it.

[Method A](#) introduces the original supervised Concept Bottleneck Model formulation, in which the final prediction is computed only from the predicted concept vector.

8.3 Concept discovery

Concept discovery methods do not begin with a complete, annotated concept vocabulary. Instead, they search a model’s learned representations for recurring clusters, directions, or patterns that may correspond to meaningful properties. A person then interprets and validates the discovered patterns.

Concept discovery is closely related to concept-based explainability. It is treated as a separate family here because the concepts are learned from representations rather than fully specified before training. A discovered pattern should not be called a meaningful concept until its interpretation is supported across multiple examples and its relationship to model behavior has been tested.

9 Taxonomy

9.1 Activation-Based Explanations

9.2 Concept-Based Explanations

TODO: One-paragraph bridge from the previous section

TODO: Define concept formally: a function, what is a concept space.

9.2.1 Preliminaries and Notation

TODO: Shared notation used by every subsection below — define once, reuse.

9.2.2 Human-Defined Concepts (Supervised)

TODO: These methods relies on human annotated concepts

9.2.2.1 Concept Bottleneck Models (CBMs)

TODO: ADD A DIAGRAMS {TRAINING AND INFERENCE}

9.2.2.1.1 training regimes

TODO: whether it is joint, sequential or indiependent

9.2.2.1.2 intervention

TODO: at inference, how does editing the concepts affect y and similar things

9.2.2.1.3 limitaions and apprpriate use cases

9.2.2.1.4 Future/emerging advancements: label free cbms?

9.2.2.1.5 key take away

9.3 Concept Discovery

9.4 Mechanistic Interpretability

9.5 Attention-Visualization Methods

10 Method A: Concept Bottleneck Models

The original Concept Bottleneck Model formulation is a supervised concept-based method built around a human-defined concept vocabulary. The broader idea of [concept-based explainability](#), including the definition of a concept and the notation used here, is introduced in the introduction.

10.1 Architecture and inference

A [Concept Bottleneck Model \(CBM\)](#) makes a prediction in two stages:

$$x \xrightarrow{g_\theta} \hat{\mathbf{c}} \xrightarrow{f_\phi} \hat{y}. \quad (10.1)$$

The concept predictor g_θ maps the input to named concept values. The task predictor f_ϕ then uses those values to predict the task output:

$$h_{\theta,\phi}(x) = f_\phi(g_\theta(x)). \quad (10.2)$$

The concept vector is called a *bottleneck* because it is the only information passed to the task predictor. In a strict CBM, there is no separate path from x to $\hat{y}$.

Each coordinate in $\hat{\mathbf{c}}$ has an assigned meaning. For a binary concept, $\hat{c}_j$ may be a hard value in $\{0, 1\}$, a score between 0 and 1, or a logit. A real-valued score gives the task predictor more graded information than a hard value. It should be interpreted as confidence only when the score has been calibrated. Categorical concepts are usually represented by one score per category, while continuous concepts can be represented directly.

The architecture of g_θ depends on the input. It may be a convolutional network for images or a transformer for text. The task predictor is often smaller. For example, a linear predictor can assign a score to class r :

$$s_r(\hat{\mathbf{c}}) = b_r + \sum_{j=1}^k w_{rj} \hat{c}_j. \quad (10.3)$$

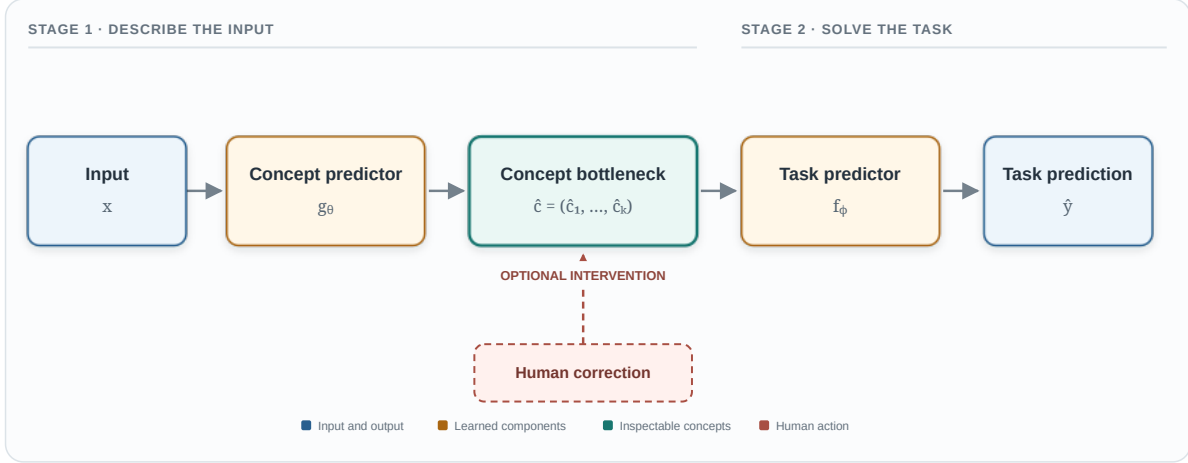


Figure 10.1: Inference and optional concept correction in a strict Concept Bottleneck Model.

Here, w_{rj} describes how concept j is associated with class r across the model, while $w_{rj}\hat{c}_j$ is that concept’s contribution to the class score for one example. Weight magnitudes can be compared directly only when the concept variables use comparable scales. These values describe the model’s calculation; they should not be interpreted as causal effects. A nonlinear task predictor can model interactions between concepts, but its decisions are harder to inspect.

10.2 What does a CBM explain?

A CBM provides a concept-level account of the final prediction. For one input, a reader can inspect:

1. **The model’s description of the input:** which concepts were predicted and their predicted scores. A score represents confidence only when it is calibrated.
2. **The task decision:** which concepts supported or opposed the predicted class. This is easiest to see with a linear task predictor.
3. **The sensitivity to corrections:** whether the task prediction changes when an incorrect concept is replaced with a corrected value.

Across a dataset, the weights of a linear task predictor show which concepts are generally associated with each class. Concept accuracy can also be examined separately for each concept to find systematic errors.

The bottleneck does not make the whole system transparent. The concept predictor g_θ may still be a complex neural network, so the CBM does not automatically explain *why* the input produced a particular concept score. It makes the second step, from concepts to the task prediction, easier to inspect.

i Example: classifying a restaurant review

Suppose a CBM predicts whether a customer recommends a restaurant. For the review “*The pasta was excellent, but we waited an hour and will not return,*” its concept predictor might produce:

Predicted concept	Score
Praises the food	0.94
Reports slow service	0.91
Intends to return	0.08

The task predictor combines these three scores to produce the recommendation label. This explanation is more informative than the label alone: it shows that the model detected both the positive comment about the food and the stronger negative signals about the overall experience. If one concept is wrong, it can be corrected and the task prediction can be recomputed.

10.3 Training regimes

Suppose the training data contain triples $(x_i, \mathbf{c}_i, y_i)$. CBMs are commonly trained in one of three ways:

1. **Independent training:** train g_θ to predict the annotated concepts, and separately train f_ϕ using the annotated concepts as input. The task predictor therefore learns from clean annotations but receives predicted concepts at inference time. See [Koh et al. \(2020\)](#), *Concept Bottleneck Models*.
2. **Sequential training:** first train g_θ , freeze it, and then train f_ϕ on $g_\theta(x_i)$. The task predictor sees the kinds of concept errors it will encounter at inference time. See [Koh et al. \(2020\)](#), *Concept Bottleneck Models*.
3. **Joint training:** train both components together using a concept loss and a task loss. The task loss is backpropagated through both f_ϕ and g_θ , while the concept loss keeps the bottleneck tied to the annotated concepts. See [Koh et al. \(2020\)](#), *Concept Bottleneck Models*.

A typical joint objective is

$$\mathcal{L} = \lambda \mathcal{L}_{\text{concept}}(g_\theta(x_i), \mathbf{c}_i) + \mathcal{L}_{\text{task}}(f_\phi(g_\theta(x_i)), y_i), \quad (10.4)$$

where λ controls the importance of accurate concept prediction. Joint training can improve task performance, but it is still necessary to evaluate whether the predicted concepts match their intended meanings.

Sheth and Kahou (2023), *Auxiliary Losses for Learning Generalizable Concept-based Models*, extend joint training with **cooperative-CBM (coop-CBM)**. Their method adds an auxiliary head that predicts the task label from the representation shared with the concept predictor. Its objective combines the concept loss, an auxiliary task loss, the final task loss, and a concept orthogonal loss. The orthogonal loss encourages within-concept similarity and separation between different concept representations. The auxiliary head helps shape the shared representation during training, while the final task predictor still receives only the predicted concepts. The paper evaluates the approach on image-classification datasets, including settings with distribution shift.

The concept loss must match the concept type. Binary concepts commonly use binary cross-entropy, categorical concepts use cross-entropy, and continuous concepts use a regression loss. If only some concepts are annotated for an example, the concept loss can be calculated over the available annotations.

10.4 Concept interventions

At inference time, a person can inspect $\hat{\mathbf{c}}$ and correct concepts that are clearly wrong. If S is the set of corrected concepts, define

$$\tilde{c}_j = \begin{cases} c_j^{\text{edit}}, & j \in S, \\ \hat{c}_j, & j \notin S. \end{cases} \quad (10.5)$$

The revised prediction is $\tilde{y} = f_\phi(\tilde{\mathbf{c}})$. This supports questions such as: *Would the prediction change if the concept “contains a negation” were corrected from absent to present?*

An intervention explains the behavior of the model under a concept edit. It does not prove that changing the corresponding property in the real world would cause the same outcome. Edits can also create unlikely combinations of concepts, so their results should be interpreted carefully.

10.5 Evaluating a CBM

Task accuracy alone is not enough. A useful evaluation should check four parts of the model:

- **Task performance:** how accurately $\hat{y}$ predicts the task label.

- **Concept performance:** how accurately each $\hat{c}_j$ predicts its annotated concept. Per-concept results are important because an average can hide a poorly learned concept.
- **Concept-set sufficiency:** how well the task can be predicted from the annotated concepts. Poor performance may indicate that the vocabulary is missing useful information, but it can also result from noisy concept annotations or an unsuitable task predictor.
- **Intervention performance:** how the task performance changes as incorrect concept predictions are corrected. This tests whether better concepts lead to better task predictions.

These measurements answer different questions. A CBM may predict the task well while predicting some concepts poorly. It may also predict every annotated concept well but perform poorly because the concept set is incomplete or the task predictor is unsuitable.

10.6 Limitations and appropriate use cases

CBMs are most useful when a task has a stable concept vocabulary, reliable concept annotations are available, and a person can meaningfully inspect or correct the concepts. They are particularly suitable when concept-level feedback is more practical than changing the original input.

Their main limitations are:

- **Incomplete vocabulary:** if important information is missing from the concept set, a strict CBM may lose predictive accuracy.
- **Annotation cost and noise:** concept labels can be expensive to collect and may differ across annotators.
- **Concept prediction errors:** an understandable concept is not useful when the model predicts it unreliably.
- **Hidden shortcuts:** the concept predictor may use an unintended input pattern to predict the correct concept value.
- **Information in soft scores:** small variations in continuous concept scores can carry information beyond the concepts' stated meanings. This can weaken interpretability and concept interventions, as discussed by [Havasi et al. \(2022\)](#).
- **Limited transparency with complex heads:** a nonlinear f_ϕ may make the relationship between concepts and predictions difficult to follow.

Architectures with residual features or a direct path from x to $\hat{y}$ may recover some accuracy. However, because named concepts no longer carry all the information used for the prediction, they do not provide the same explanation guarantee as a strict CBM.

10.7 Reducing the need for concept annotations

Recent extensions reduce the amount of per-example concept labeling. [Label-free CBMs](#) use a chosen list of concept names together with pretrained representations to construct a concept bottleneck without manually labeling every training example. [Post-hoc CBMs](#) project a pretrained model's representations into a concept space rather than training the entire model from the beginning.

The term *label-free* refers to concept labels: these methods can still require task labels, a human-selected concept vocabulary, and pretrained models. Their concept scores must also be validated. A concept name alone does not guarantee that the score consistently measures the intended property.

Key takeaway

A strict CBM predicts named concepts first and makes the final prediction only from those concepts. This structure makes concept inspection and correction possible. The explanation is useful only when the concept vocabulary is relevant and the predicted concept values are reliable.

11 Method(s) Page 2

12 ...

13 Applications

14 Conclusion

Part III

Explainable Reinforcement Learning

15 Introduction to Explainable RL

Note to self:

Before:

- Foundations and taxonomy
 - split into two separate pages
- Visualization-based approaches
 - mentioned in the text, doesn't need its own section
- Policy extraction and decision trees
 - policy-level XRL, okay
- Explaining decisions through critical states
 - hmm, feature importance?
- Trajectory-based methods
 - MDP and/or policy-level
- Explainability via counterfactual reasoning
- Evaluation of XRL methods

15.1 Foundations

There's value-based and policy-based RL.

Why is it important? For example, deploying RL-based algorithms in finance requires algorithmic transparency to comply with regulations; in sequential human-AI collaboration process, users need to understand agent behavior to trust and cooperate.

When it comes to RL, many things are desirable: fast learning (i.e., sample efficiency), high final performance, safety guarantees, transparent behavior processes, etc. To keep this tutorial focused, we consider constrained RL formulations for safe RL out of scope.

15.2 Challenges in XRL

- large number of decisions over a period of time
- actions are not independent of each other -> explanation can't be either
- no static dataset, agent-generated data

16 Taxonomy

Diverse approaches exist to explain RL agents, most of which can be categorized as intrinsic vs. post-hoc, as commonly done for supervised learning. However, in this tutorial we follow the taxonomy by Milani et al. (2024), because it applies directly to the unique setting of RL.

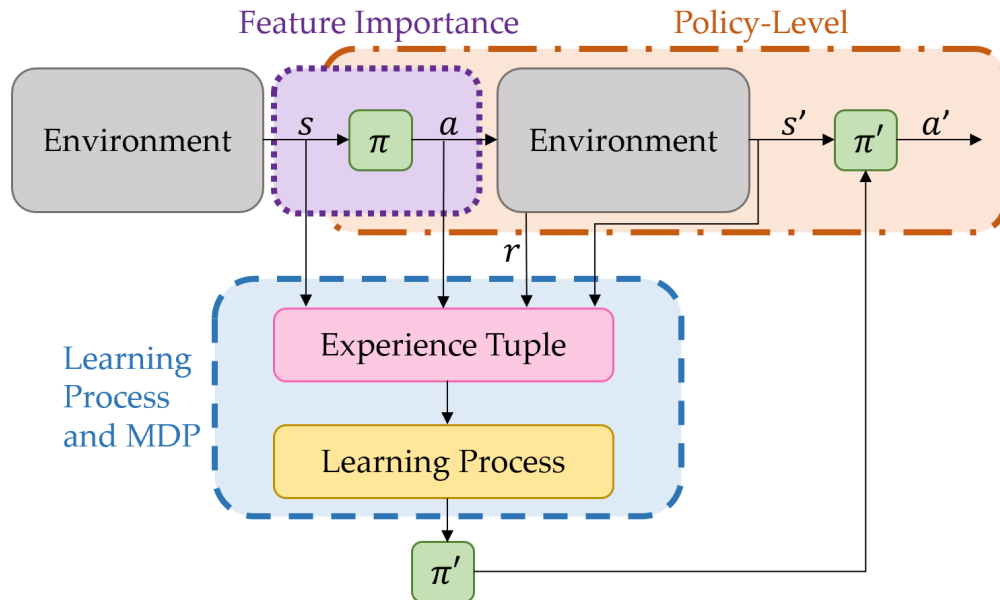


Figure 16.1: Taxonomy by Milani et al., 2024

16.1 Feature importance (or: action level)

This approach works on low-level states, explaining what is important for finding the right actions to explain a specific state. This includes saliency maps.

- feature importance
- expected outcomess

16.2 Learning process and MDP level (or: sequency level)

This shows the past experiences or the components of the MDP that led to the current behavior; it includes decision trees and finite state machines.

- counterfactual sequency
- important elements
- human-readable MDP

16.3 Policy level

The vast majority (175 works, whereas sequence-level is only 11) Aiming to explain an agent's long-term behavior, this includes reward decomposition and counterfactuals.

- constructing interpretable policies
 - surrogate model
 - inherently understandable
- summarizing policies
- human-readable MDP
 - surrogate model
 - inherently understandable
- Visual anlysis

Other taxonomies:

- Following the general XAI taxonomy, XRL methods can also be divided by scope (local vs. global) and time of extraction (intrinsic, which is always model-specific, vs. post-hoc, which can be model-specific or -agnostic). Bekkemoen (2024) provide a useful taxonomy by further subdividing intrinsic explainability and post-hoc explainability into explanations via generation, representation, and inspection.
- Interpretable inputs can be achieved via structured approaches, extracting symbolic representations, hierarchical approaches {Glanois et al. (2024)}
- Qing et al. (2022) divides XRL methods into four categories: agent model-explaining, reward-explaining, state-exlaining, task-explaining.
- Dazeley, Vamplew, and Cruz (2023) propose the causal XRL framework (CXF), a wholistic cognitive model for XAI, which uses RL as the backbone. While this presents an ambitious and inspiring perspective for future work, it is not a clear-cut taxonomy for XRL methods.

16.4 Alternative taxonomy structure:

16.4.1 Visualization-Based Approaches

State-value heatmaps can reveal an agent’s strategic understanding of the environment. This is particularly useful in navigation-based settings such as mazes.

For more fine-grained explanations, we can also visualize state-action values (Q values) to identify the agent’s understanding of actions from the perspective of a given state.

Visualization-based explainability toolkits for RL have been proposed such as DQNViz (Junpeng Wang et al. 2018), developed for DQN-based agents learning Atari games.

16.4.2 More stuff to sort in

Decision trees are inherently interpretable. Once a policy is learned, the reasoning behind every decision can be clearly traced by following the tree’s decision nodes. This works well when individual state features are meaningful (e.g., position, velocity); however, it struggles with high-dimensional state spaces such as raw pixels

Example: VIPER Bastani, Pu, and Solar-Lezama (2018), an approach to make RL verifiable by learning decision tree policies

16.5 Representation space analysis

Just like in supervised learning, it is of great interest to understand what kind of internal representations deep networks actually discover. High-dimensional state spaces such as pixels can be mapped to lower-level features using t-SNE.

16.6 Explaining decisions through critical states

Example: StateMask ?

16.7 Trajectory-based methods

An example of behavior discovery and attribution is this: ?

16.8 Explainability via counterfactual reasoning

A recent focus in the RL community is using world models for better sample-efficiency and higher performance. By having a world model learn the environment dynamics $P(s_{t+1} \mid s_t, a_t)$, we can predict future states without interaction with the real environment. This is essential for planning and also has the useful side effect that it enables counterfactual reasoning, useful for generating “what-if” scenarios.

? is among the first to use world models for counterfactual explanations in RL.

? uses backward counterfactuals (“What state would cause agent to do X ?”), using a reverse world model $P(s_t \mid s_{t+1}, a_t)$

16.9 Evaluation of XRL methods

Unique challenges to XRL are the sequential nature of decisions, requiring understanding of strategies rather than single model predictions, temporal dependencies making explanations more complex, and evolving agent behavior during learning.

There are three evaluation levels identified by ?, which are increasingly specific and costly: functionally grounded (computational metrics only), human-grounded (general users + simplified tasks), and application-grounded (domain experts + real tasks).

16.10 Applications

17 Perturbation-Based Saliency Maps for RL

Key reference: Greydanus et al. (2018) — [paper](#) and [code](#)

💡 TL;DR

Perturbation-based saliency maps are useful to visualize the “attention” of an RL agent in a visual environment.

Introduction

Greydanus et al. (2018) use saliency maps to visualize RL policies in pixel-based environments and evaluate their framework on Atari 2600 games. In particular, they use a perturbation-based approach, with the leading question: “How much does removing information from the region around location (i, j) change the policy?” The saliency maps can be applied at different stages: (1) during training to understand how an agent learns and (2) after training, observing what parts of the state space the agent’s final policy attends to.

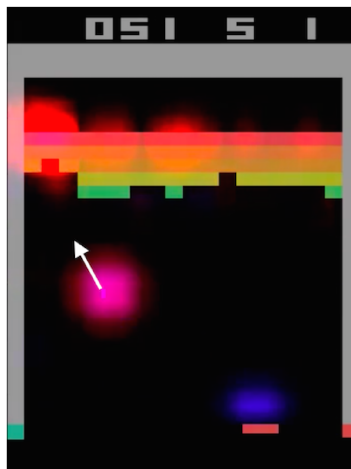


Figure 17.1: Tunneling strategy in Atari Breakout

Method: Perturbation-based saliency maps

The core idea behind this method is to measure how much a neural network’s output (its policy and value estimates) changes when information is selectively removed from the input image. Rather than simply blacking out a region, the method introduces **spatial uncertainty** by

locally blurring the image. If the agent’s proposed action or value estimate drastically changes when a specific area is blurred, that region is highly salient to the agent’s decision-making process.

The perturbation applied to a localized coordinate $((i,j))$ on an image frame I_t is defined by the following key math:

$$\Phi(I_t, i, j) = \underbrace{I_t}_{\text{image frame}} \odot (1 - \underbrace{M(i, j)}_{\text{mask}}) + \underbrace{A(I_t, \sigma_A)}_{\text{blurred frame}} \odot M(i, j),$$

where $\odot$ denotes the Hadamard product and $I_{1:t}$ is a sequence of images.

This visualization technique is separate from the RL algorithm used for training. The authors used the A3C algorithm, which yields both a policy and value function. The above procedure for constructing pertubation-based saliency was applied to both the policy π and V^π . They use patches of 5x5 pixels to reduce the required computation.

Tradeoffs

	<u>Cons</u>
Pros	
Easy implementation	Requires running the model multiple times to test different image locations
Generally applicable to any pixel-based state spaces	Gives no insights into the weights; model remains black-box

Related

- Instead of pertubation-based saliency maps, prior work has used Jacobian-based ones (Z. Wang et al. 2016; Zahavy, Ben-Zrihem, and Mannor 2016).

18 Method(s) Page 2

19 ...

20 Applications

21 Conclusion

Part IV

Explainability in Multi-Agent Systems

22 Introduction

Explainable multi-agent systems (XMAS) refers to the subfield of artificial intelligence that integrates LLM explainability methods within multi-agent AI systems (MAS). This recently established field of study addresses the emergent behaviours of MAS, which derive from the added complexity of multi-agent settings. As single-agent AI systems become progressively more commercialized (finance, e-commerce, legal services), the natural step towards MAS necessitates the corresponding MAS-specific explainability methods.

23 Taxonomy

23.1 What are agents?

Generally, agents refer to *any entity capable of perceiving its environment and taking action* (Cheng et al. 2024). Namely, large language model (LLM)-based agents leverage the natural language processing capabilities of LLMs (complex reasoning, text summarization, machine translation) % <https://link.springer.com/article/10.1007/s11704-025-50472-3> , while attempting to address their fundamental limitations:

% <https://www.jair.org/index.php/jair/article/view/18675/27253> % <https://dl.acm.org/doi/abs/10.1145/3787470>

1. **Hallucination.** The tendency of LLMs to produce *plausible yet factually unsupported content*, deriving from data misinformation, pre-training biases, over-confidence, among others. %<https://dl.acm.org/doi/pdf/10.1145/3703155>
2. **Training data.** LLM intelligence is limited by the data available during training.
3. **Reasoning.** Although LLMs continue to impress in natural language processing, model reasoning remains a well-reported challenge.
4. **Prompt Engineering.** LLMs are sensitive to minor details in prompts, making prompt improvement a tedious, manual task. % <https://www.jair.org/index.php/jair/article/view/18675/27253>

% <https://arxiv.org/abs/2308.11432> Consequently, the architecture of LLM-based agents adds four fundamental modules absent from “pure LLMs”: - Profiling: Defining agent identity - Memory: Storing experience - Planning: Decomposing goals - Action: Executing plans via tools or internal knowledge

% <https://link.springer.com/article/10.1007/s44336-024-00009-2> Some authours argue for a fifth module, evolution, describing how agents progressively enhance their intelligence through self-reflection.

Analogously, agents can be understood as *proactive* whereas LLMs are *reactive*. Alternatively, LLMs *predict*, and agents *reason*, *act*, and *interact*. % <https://www.jair.org/index.php/jair/article/view/18675/27253> % confirm claim.

Ideally, the foundational agent architecture would address the concerns of LLMs through ... % <https://www.jair.org/index.php/jair/article/view/18675>

Implementations of single-agent systems can be used in ... % <https://arxiv.org/abs/2512.08296>

Nonetheless, single-agent systems have key pitfalls... % <https://arxiv.org/abs/2511.01149>

23.2 What are multi-agent systems?

Multi-agent systems are systems that enclose multiple autonomous LLM-based agents to enable more complex reasoning and simulation. Leveraging the foundational “wisdom of the multitude” is just a natural step in the evolution of agentic AI, addressing the pitfalls of single-agent systems as listed above.

% <https://arxiv.org/pdf/2402.01680> % Politics - aristotle

Specifically, multi-agent systems surpass single-agent systems in... % <https://arxiv.org/abs/2511.01149>
(same as above)

Multi-agent systems are used in ... % couple examples

As the implementations of multi-agent systems become more integrated into society, the need for explainability methods is just as required. % tie back to explainability

23.3 Technical Approaches and Methodologies

% Single-agent based approaches (but in to multi-agent contexts)

% SAEs, linear probes, logit/tuned lenses, etc.

% Multi-agent based approaches

% describe how certain behaviours occur ONLY in a multi-agent context: collusion, complex scheming

% include black-box tools (LLM-as-a-judge, human-in-the-loop)#

24 Method(s) Page 1

25 Method(s) Page 2

26 ...

27 Applications

28 Conclusion

References

- Arrieta, Alejandro Barredo, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador García, et al. 2019. “Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges Toward Responsible AI.” <https://arxiv.org/abs/1910.10045>.
- Bastani, Osbert, Yewen Pu, and Armando Solar-Lezama. 2018. “Verifiable Reinforcement Learning via Policy Extraction.” *Advances in Neural Information Processing Systems* 31.
- Bekkemoen, Yanzhe. 2024. “Explainable Reinforcement Learning (XRL): A Systematic Literature Review and Taxonomy.” *Machine Learning* 113 (1): 355–441.
- Breiman, Leo, Jerome H Friedman, Richard A Olshen, and Charles J Stone. 1984. *Classification and Regression Trees*. Wadsworth & Brooks/Cole Advanced Books & Software.
- Cheng, Yuheng, Ceyao Zhang, Zhengwen Zhang, Xiangrui Meng, Sirui Hong, Wenhao Li, Zihao Wang, et al. 2024. “Exploring Large Language Model Based Intelligent Agents: Definitions, Methods, and Prospects.” *arXiv Preprint arXiv:2401.03428*.
- Dazeley, Richard, Peter Vamplew, and Francisco Cruz. 2023. “Explainable Reinforcement Learning for Broad-Xai: A Conceptual Framework and Survey.” *Neural Computing and Applications* 35 (23): 16893–916.
- Glanois, Claire, Paul Weng, Matthieu Zimmer, Dong Li, Tianpei Yang, Jianye Hao, and Wulong Liu. 2024. “A Survey on Interpretable Reinforcement Learning.” *Machine Learning* 113 (8): 5847–90.
- Greydanus, Samuel, Anurag Koul, Jonathan Dodge, and Alan Fern. 2018. “Visualizing and Understanding Atari Agents.” In *International Conference on Machine Learning*, 1792–1801. PMLR.
- Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference and Prediction*. 2nd ed. springer.
- Lundberg, Scott, and Su-In Lee. 2017. “A Unified Approach to Interpreting Model Predictions.” <https://arxiv.org/abs/1705.07874>.
- Milani, Stephanie, Nicholay Topin, Manuela Veloso, and Fei Fang. 2024. “Explainable Reinforcement Learning: A Survey and Comparative Review.” *ACM Computing Surveys* 56 (7): 1–36.
- Miller, Tim. 2019. “Explanation in Artificial Intelligence: Insights from the Social Sciences.” *Artificial Intelligence* 267: 1–38. <https://doi.org/https://doi.org/10.1016/j.artint.2018.07.007>.
- Minh, Anh Pham Thi. 2023. “Overview of Class Activation Maps for Visualization Explainability.” <https://arxiv.org/abs/2309.14304>.

- Molnar, Christoph. 2025. *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 3rd ed. <https://christophm.github.io/interpretable-ml-book>.
- Qing, Yunpeng, Shunyu Liu, Jie Song, Yang Zhou, Kaixuan Chen, Huiqiong Wang, and Mingli Song. 2022. “A Survey on Explainable Reinforcement Learning: Concepts, Algorithms, Challenges.” *arXiv Preprint arXiv:2211.06665*.
- Quinlan, J. R. 1986. “Induction of Decision Trees.” *Machine Learning* 1 (1): 81–106. <https://doi.org/10.1007/BF00116251>.
- Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin. 2016. “”Why Should i Trust You?”: Explaining the Predictions of Any Classifier.” <https://arxiv.org/abs/1602.04938>.
- Ruder, Sebastian. 2017. “An Overview of Gradient Descent Optimization Algorithms.” <https://arxiv.org/abs/1609.04747>.
- Selvaraju, Ramprasaath R., Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. 2019. “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization.” *International Journal of Computer Vision* 128 (2): 336–59. <https://doi.org/10.1007/s11263-019-01228-7>.
- Simonyan, Karen, Andrea Vedaldi, and Andrew Zisserman. 2014. “Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps.” <https://arxiv.org/abs/1312.6034>.
- Wang, Jingyuan, and Jiahao Ji. 2025. “A Tutorial on Regression Analysis: From Linear Models to Deep Learning – Lecture Notes on Artificial Intelligence.” <https://arxiv.org/abs/2512.04747>.
- Wang, Junpeng, Liang Gou, Han-Wei Shen, and Hao Yang. 2018. “DNQViz: A Visual Analytics Approach to Understand Deep q-Networks.” *IEEE Transactions on Visualization and Computer Graphics* 25 (1): 288–98.
- Wang, Ziyu, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas. 2016. “Dueling Network Architectures for Deep Reinforcement Learning.” In *International Conference on Machine Learning*, 1995–2003. PMLR.
- Zahavy, Tom, Nir Ben-Zrihem, and Shie Mannor. 2016. “Graying the Black Box: Understanding DQNs.” In *International Conference on Machine Learning*, 1899–1908. PMLR.